CONFIDENTIAL · SECURITY AUDIT

VERSION 2 · RE-REVIEW

Security Audit
Tech for Palestine
Website

techforpalestine.org · Cloudflare Pages & Workers

This is a **delta re-review** following security patches applied on 2026-04-02. Three critical findings from v1 were addressed; this report documents two new issues introduced by those patches, confirms remaining open findings, and updates the remediation priority list accordingly.

ORIGINAL AUDIT	RE-REVIEW DATE	FILES REVIEWED
2026-03-31	2026-04-02	qgiv-webhook.ts · membership-invite.ts · wrangler configs + all v1 scope

Executive Summary — v2 Re-review

Delta audit of patches applied after v1 report, plus carryover open findings.

1	6	7	3
CRITICAL (NEW)	HIGH (1 NEW)	MEDIUM (1 NEW)	LOW (UNCHANGED)

Three critical findings from v1 were addressed. However, the authentication mechanism chosen for the webhook endpoint has introduced a **new critical vulnerability**: the secret is passed as a URL query parameter, which means it will appear in plaintext in Cloudflare access logs, proxy logs, and any log drain. **Treat `QGIV_WEBHOOK_SECRET` as already compromised and rotate it immediately.**

All other findings from v1 remain open and unaddressed.

PATCH ASSESSMENT — V1 FINDINGS

ID	Finding	v1 Severity	Status	Notes
C-1	API key hardcoded in wrangler.toml	CRITICAL	Fixed	Key removed; git history still contains it — rotate
C-1b	CF account ID in worker wrangler.toml	CRITICAL	Fixed	Removed from file
C-2	Unauthenticated membership invite endpoint	CRITICAL	Partial	Bearer auth added; timing-safe comparison missing (N-2); body still unvalidated (N-3)
C-3	Unverified webhook triggers privileged invite	CRITICAL	Partial	Secret check added but secret is in URL — introduces new critical N-1
H-1 → H-5	Console logging, caching, rate limiting, CORS	HIGH	Open	Not addressed
M-1 → M-6	CSRF, input validation, headers, globalThis, etc.	MEDIUM	Open	Not addressed

ID	Finding	v1 Severity	Status	Notes
L-1 → L-3	Notion IDs in config, SRI, image proxy auth	LOW	Open	Not addressed

FULL ISSUE REGISTER — V2

ID	Finding	Severity	OWASP	Status
N-1	Webhook secret exposed in URL query parameter	NEW CRITICAL	A02 – Cryptographic Failures	New
N-2	Timing attack on token comparison (both endpoints)	NEW HIGH	A02 – Cryptographic Failures	New
N-3	membership-invite body still forwarded without validation	NEW MEDIUM	A03 – Injection	New
H-1	Secret key logged to browser console (api.ts)	HIGH	A09 – Logging Failures	Open
H-2	Full PII payload echoed in HTTP response	HIGH	A02 – Cryptographic Failures	Open
H-3	Middleware caches POST/API responses as public	HIGH	A05 – Misconfiguration	Open
H-4	No rate limiting on any endpoint	HIGH	A04 – Insecure Design	Open
H-5	Wildcard CORS on all API endpoints	HIGH	A05 – Misconfiguration	Open
M-1	No CSRF protection on form endpoints	MEDIUM	A01 – Broken Access Control	Open
M-2	No input length/format validation on forms	MEDIUM	A03 – Injection	Open
M-3	getEnv falls back to globalThis property lookup	MEDIUM	A03 – Injection	Open
M-4	Verbose logging of S3 signed URLs and payloads	MEDIUM	A09 – Logging Failures	Open
M-5	Internal error.message returned in HTTP responses	MEDIUM	A05 – Misconfiguration	Open
M-6	Missing security headers (CSP, X-Frame-Options, etc.)	MEDIUM	A05 – Misconfiguration	Open

ID	Finding	Severity	OWASP	Status
L-1	Notion DB IDs in wrangler.toml [vars]	LOW	A05 – Misconfiguration	Open
L-2	External payment script without SRI hash	LOW	A08 – Integrity Failures	Open
L-3	Image proxy has no authentication	LOW	A01 – Broken Access Control	Open

NEW FINDINGS INTRODUCED BY V2 PATCH

NEW CRITICAL N-1

Webhook Secret Exposed in URL Query Parameter — Logged by Cloudflare

File: src/pages/api/qgiv-webhook.ts:14–17 **OWASP:** A02:2021 – Cryptographic Failures

The webhook is now authenticated, but the secret is read from the URL `?secret=TOKEN` query parameter instead of an HTTP header. This appears to authenticate the endpoint, but it fundamentally undermines the confidentiality of the secret.

```
// qgiv-webhook.ts:14-17
const url = new URL(request.url);
const token = url.searchParams.get("secret");  // ← secret is IN the URL
```

WHY THIS IS A CRITICAL REGRESSION

URLs are treated as non-sensitive data by every layer of the HTTP stack. They are logged by default in:

- **Cloudflare access logs** — every request URL is stored in Cloudflare's dashboard and any configured log drain
- **Server-side access logs** — standard access log format includes the full request URL
- **Browser history** — if anyone pastes this URL in a browser or documentation
- **Referer headers** — if the webhook URL is ever linked from a page
- **Any monitoring, APM, or SIEM tool** that receives access logs

⚠ TREAT AS COMPROMISED

`QGIV_WEBHOOK_SECRET` must be considered fully compromised and rotated immediately. Audit Cloudflare access logs for the period since this change was deployed to determine if the secret has been observed.

ATTACK SCENARIO

Any person with access to Cloudflare dashboard, a log drain, or any proxy between QGiv and the server can extract the secret from logs and forge arbitrary webhook events — restoring the original C-3 vulnerability.

SECURE FIX — USE AN HTTP HEADER INSTEAD

```
// qgiv-webhook.ts — replace lines 14-17
// QGiv supports a custom header field in its webhook configuration
const token = request.headers.get("X-Webhook-Secret");

if (!webhookSecret || !token || !constantTimeEqual(token, webhookSecret)) {
  return new Response(JSON.stringify({ message: "Unauthorized" }), {
    status: 401,
    headers: { "Content-Type": "application/json" },
```

```
  });
}
```

Configure QGiv to send `X-Webhook-Secret: <secret>` as a custom header on every webhook delivery. HTTP headers are never included in access log formats by default.

NEW HIGH N-2

Timing Attack on Token Comparison — Both Endpoints

Files: src/pages/api/membership-invite.ts:12 · src/pages/api/qgiv-webhook.ts:17

OWASP: A02:2021 – Cryptographic Failures

Both endpoints use JavaScript's `!==` operator to compare the supplied token against the expected secret. JavaScript string comparison short-circuits on the first differing byte, causing measurably different execution times depending on how many leading characters match.

```
// membership-invite.ts:12 — timing-unsafe
if (!inviteSecret || !token || token !== inviteSecret)

// qgiv-webhook.ts:17 — timing-unsafe
if (!webhookSecret || !token || token !== webhookSecret)
```

ATTACK SCENARIO

An attacker sends thousands of requests with candidate tokens varying the first character, measures average response time per candidate, and identifies which character produces a longer match (more equal bytes before short-circuit). Repeating this for each position enumerates the full secret without ever seeing it directly. This is a classical timing oracle attack against HMAC/token comparison.

SECURE FIX — CONSTANT-TIME COMPARISON

```
// Add this helper (shared utility or inline)
import { timingSafeEqual } from "node:crypto";

function constantTimeEqual(a: string, b: string): boolean {
  // Length check must not reveal which is longer — check both ways
  if (a.length !== b.length) return false;
  const aBuf = Buffer.from(a, "utf8");
  const bBuf = Buffer.from(b, "utf8");
  return timingSafeEqual(aBuf, bBuf);
}

// Usage — replaces !== in both files
if (!inviteSecret || !token || !constantTimeEqual(token, inviteSecret)) { ... }
```

Note: `node:crypto` is available in the Cloudflare Pages runtime because `nodejs_compat` is already enabled in `wrangler.toml`.

NEW MEDIUM N-3

membership-invite Still Proxies Arbitrary Body to Hub API Without Validation

File: src/pages/api/membership-invite.ts:36–43 **OWASP:** A03:2021 – Injection

Authentication is now required to reach this endpoint, but an authenticated caller can still send arbitrary JSON fields that are forwarded verbatim to the Hub API. If the Hub API is vulnerable to mass-assignment or parameter injection, a caller with the invite secret could set unauthorized fields — for example, escalating an invite to an admin role by adding extra properties.

```
// membership-invite.ts:36-43
const upstream = await fetch(`${hubApiUrl}/api/auth/invite`, {
  method: "POST",
  headers: { Authorization: `Bearer ${hubApiKey}` },
  body: JSON.stringify(body),  // ← entirely caller-controlled JSON
});
```

SECURE FIX — VALIDATE AND WHITELIST FIELDS BEFORE FORWARDING

```
const { email, type } = body as Record<string, unknown>;

if (typeof email !== "string" || !/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email)) {
  return new Response(JSON.stringify({ message: "Invalid email" }), { status: 40
0 });
}
if (type !== "paid" && type !== "free") {
  return new Response(JSON.stringify({ message: "Invalid type" }), { status: 400
});
}

// Forward ONLY the validated, whitelisted fields
body: JSON.stringify({ email, type })
```

REMAINING OPEN FINDINGS — HIGH SEVERITY

These findings were reported in v1 and have not been addressed. Detail is provided for completeness.

HIGH H-1 PUBLIC_SECRET_KEY Logged to Browser Console at Module Load

File: src/store/api.ts:7–8 **Page:** /project-apply (client-side React bundle)

OWASP: A09:2021 – Security Logging Failures

`api.ts` runs client-side. Both `PUBLIC_API_URL` and `PUBLIC_SECRET_KEY` are printed to the browser console on every page load of `/project-apply`. The key is also embedded as a plain `Authorization` header in every form submission Axios request — visible to any user in DevTools → Network tab.

```
// api.ts:7-8 — runs in every visitor's browser
console.log("PUBLIC_API_URL:", API_URL);
console.log("PUBLIC_SECRET_KEY:", SECRET_KEY);  // visible in DevTools
```

FIX

Remove both `console.log` calls. Proxy authenticated API requests through a server-side Astro API route that holds the credential exclusively server-side, never exposing it in the browser bundle.

HIGH H-2 Full Donor PII Payload Echoed in HTTP Response Body

File: src/pages/api/qgiv-webhook.ts:166–178 **OWASP:** A02:2021 – Cryptographic Failures

When donation type cannot be determined, the full unredacted webhook payload — containing donor name, email, postal address, and transaction ID — is returned verbatim in the HTTP response body. The payload is also logged in full on line 27 (`console.log("QGiv webhook received:", JSON.stringify(payload, null, 2))`).

```
// qgiv-webhook.ts:166-178 — PII in response
return new Response(JSON.stringify({
  success: true,
  message: "Webhook received but donation type unclear",
  payload,  // ← full donor record returned to caller
}), ... );
```

FIX

Return only `{"success": true}` . Log a redacted version server-side (strip email, name, address before logging).

HIGH H-3 Global Middleware Caches POST and API Responses as "public"

File: src/middleware/index.ts:6 **OWASP:** A05:2021 – Security Misconfiguration

The global middleware sets `Cache-Control: public, max-age=600` on every response with no filtering by method or path. POST responses from `/api/endorsement-request` and `/api/e4p-pledge-sign` get this header. A CDN or proxy could cache a 201 response and serve it to subsequent callers, silently discarding their form submissions.

FIX

```
const isApi = context.url.pathname.startsWith("/api/");
const isGet = context.request.method === "GET";
response.headers.set("Cache-Control",
  !isGet || isApi ? "no-store" : "public, max-age=600"
);
```

HIGH H-4

No Rate Limiting on Any Endpoint

File: All src/pages/api/* endpoints **OWASP:** A04:2021 – Insecure Design

Every write endpoint — including those writing to Notion databases, sending emails, and triggering Hub invitations — has zero rate limiting. A single attacker IP can flood `/api/endorsement-request` or `/api/e4p-pledge-sign` to fill Notion with spam at full throughput.

> **FIX**
>
> Add Cloudflare WAF rate limiting rules (available on the free tier): 10 POST requests per 60 seconds per IP on `/api/*`. Add honeypot hidden fields to all public forms.

HIGH H-5

Wildcard CORS (Access-Control-Allow-Origin: *) on All Endpoints

File: Every file in src/pages/api/ **OWASP:** A05:2021 – Security Misconfiguration

Every API endpoint returns `Access-Control-Allow-Origin: *`. Combined with missing CSRF protection (M-1), any malicious third-party website can make cross-origin AJAX requests to write endpoints on behalf of a visiting user, submitting forms or triggering actions without their knowledge.

> **FIX**
>
> Replace with `"Access-Control-Allow-Origin": "https://techforpalestine.org"` on all write endpoints. Wildcard CORS is acceptable only for explicitly public read-only data APIs (GET /api/events, etc.).

REMAINING OPEN FINDINGS — MEDIUM SEVERITY

MEDIUM M-1 No CSRF Protection on State-Mutating Form Endpoints

Files: src/pages/api/endorsement-request.ts · src/pages/api/e4p-pledge-sign.ts

OWASP: A01:2021 – Broken Access Control

POST endpoints accept requests without verifying the origin. Combined with H-5 (wildcard CORS), any malicious website can silently submit these forms on a victim's behalf. **Fix:** Verify the `Origin` request header equals `https://techforpalestine.org` on all POST endpoints before processing.

MEDIUM M-2 No Input Length or Format Validation on Form Endpoints

Files: src/pages/api/endorsement-request.ts · src/pages/api/e4p-pledge-sign.ts

OWASP: A03:2021 – Injection

Field presence is checked, but there is no validation of email format, URL format (website, campaign link, LinkedIn accept any string including `javascript:` URIs), or string length. A 10 MB `campaignPurpose` will be accepted and written to Notion. **Fix:** Validate email with regex, validate URLs with `new URL()`, enforce max length on all text fields.

MEDIUM M-3 getEnv Falls Back to Unsafe globalThis Property Lookup

File: src/utils/getEnv.ts:24–26 **OWASP:** A03:2021 – Injection

The final fallback reads `(globalThis as any)[name]`. If any third-party dependency or prototype pollution attack sets `globalThis.NOTION_SECRET`, it will be used as the API credential. **Fix:** Remove the `globalThis` fallback entirely; the Cloudflare runtime env and `import.meta.env` are sufficient.

MEDIUM M-4

Verbose Logging of AWS S3 Signed URLs, Emails, and Payloads

Files: src/store/notionClient.ts:45–68 · src/pages/api/qgiv-webhook.ts:27, 51, 60

OWASP: A09:2021 – Security Logging Failures

Multiple `console.log` calls emit full AWS S3 signed URLs (which contain `X-Amz-Credential` and `X-Amz-Signature` in query parameters), donor email addresses, and full QGiv payloads. In any log aggregation system, these constitute ongoing PII and credential leakage. **Fix:** Remove all debug `console.log` calls from production paths.

MEDIUM M-5

Internal Error Details Returned in HTTP Response Bodies

File: src/pages/api/qgiv-webhook.ts:151, 185 **OWASP:** A05:2021 – Security Misconfiguration

Exception `.message` strings from fetch failures are included in HTTP error responses: `details: error.message`. These may reveal internal hostnames, library internals, or stack information useful for reconnaissance. **Fix:** Log full errors server-side with a correlation ID; return only `{"error": "An internal error occurred"}` to callers.

MEDIUM M-6

Missing Security Headers (CSP, X-Frame-Options, Referrer-Policy, etc.)

File: astro.config.mjs · src/middleware/index.ts **OWASP:** A05:2021 – Security Misconfiguration

No security headers are set anywhere. The absence of a Content Security Policy is the most significant gap — it provides no browser-level mitigation against XSS or inline script injection.

```
# public/_headers — simplest approach for Cloudflare Pages
/*
  X-Content-Type-Options: nosniff
  X-Frame-Options: DENY
  Referrer-Policy: strict-origin-when-cross-origin
  Permissions-Policy: camera=(), microphone=(), geolocation=()
  Content-Security-Policy: default-src 'self';
    script-src 'self' https://secure.qgiv.com;
    img-src 'self' data: https:;
    connect-src 'self' https://plausible.io;
```

REMAINING OPEN FINDINGS — LOW SEVERITY

LOW L-1 Notion Database IDs Remain in wrangler.toml [vars]

File: wrangler.toml:10–16

Seven Notion database IDs are still committed in the `[vars]` block of the tracked `wrangler.toml`. While not standalone credentials, they assist an attacker who has separately obtained the Notion API secret. Move to Cloudflare secrets or a `.env` file excluded from git.

LOW L-2 QGiv Payment Script Loaded Without Subresource Integrity Hash

File: src/components/MembershipPage.tsx **OWASP:** A08:2021 – Software & Data Integrity Failures

`https://secure.qgiv.com/resources/core/js/embed.js` is dynamically injected without an `integrity` attribute. A compromised QGiv CDN or a network-level interception could deliver malicious JavaScript directly into the payment context. Add an `integrity="sha384-…"` attribute computed from a known-good version of the script.

LOW L-3 Notion Image Proxy Has No Authentication or Access Control

File: cloudflare-worker/worker.js **OWASP:** A01:2021 – Broken Access Control

The worker validates that decoded URLs point to Notion S3 domains (preventing open-relay abuse), but any actor can use the proxy to cache and serve any valid Notion S3 URL they possess, potentially bypassing Notion's own access expiry for content they should no longer have access to. Low severity given images are public via Notion's CDN already.

GOOD PRACTICES INTRODUCED IN V2 PATCH

- **Authentication is now present** on both the webhook and membership-invite endpoints — a significant improvement in the right direction.
- **Proper HTTP 401 responses** with JSON bodies returned on auth failure — no information leakage on rejection.
- **Cloudflare runtime env bindings respected first** (`locals.runtime.env`) before `import.meta.env` fallback — correct production precedence.
- **`PROJECTHUB_API_KEY` removed from `wrangler.toml`** — resolves the most impactful v1 finding (note: git history still contains it; rotate the key).
- **Cloudflare account ID removed** from the worker wrangler config.

UPDATED PRIORITISED REMEDIATION PLAN

Priority	Action	ID	Effort
Immediate	Rotate `QGIV_WEBHOOK_SECRET` — it has been in URL query params and is in Cloudflare logs	N-1	Minutes
Immediate	Move webhook auth from `?secret=` query param to `X-Webhook-Secret` HTTP header	N-1	30 min
Immediate	Rotate `PROJECTHUB_API_KEY` — still in git history from v1	C-1	Minutes
Day 1	Replace `!==` token comparison with `timingSafeEqual` in both auth endpoints	N-2	1 hour
Day 1	Add field whitelist and validation to `membership-invite.ts` before forwarding body	N-3	1 hour
Day 1	Remove `console.log` of secrets and PII from `api.ts` , `notionClient.ts` , `qgiv-webhook.ts`	H-1, M-4	30 min
Day 1	Stop echoing `payload` in webhook fallthrough response and error details in 500s	H-2, M-5	30 min
Week 1	Fix global middleware to apply `no-store` to POST and API responses	H-3	30 min

Priority	Action	ID	Effort
Week 1	Implement Cloudflare WAF rate limiting on all POST endpoints (10 req / 60 s / IP)	H-4	2 hours
Week 1	Restrict CORS to `https://techforpalestine.org` on all write endpoints	H-5	1 hour
Week 1	Add security headers via `public/_headers` or middleware (CSP, X-Frame-Options, etc.)	M-6	1 hour
Week 2	Add email format, URL format, and length validation on all form POST endpoints	M-2	3 hours
Week 2	Add `Origin` header verification for CSRF mitigation on all POST endpoints	M-1	1 hour
Week 2	Remove `globalThis` property-name fallback from `getEnv`	M-3	15 min
Week 2	Add SRI integrity hash to QGiv embed script; move Notion DB IDs to CF secrets	L-1, L-2	1 hour